

Telemetry Yield na NVIDIA A40: CUDA i kierunki rozwoju algorytmu

1. Wnioski

Przeniesienie istotnej części Telemetry Yield na A40 jest technicznie uzasadnione. Najlepszym pierwszym krokiem nie jest zamiana całego programu w sieć neuronową, lecz oddzielny backend GPU, który przetwarza jednocześnie wiele istniejących hipotez. Dopiero po jego kwalifikacji warto wykorzystać dodatkowy budżet na dokładniejszy model kanału i detekcję z miękkim wyjściem.

Trzeba rozdzielić trzy cele: skrócenie czasu identycznych obliczeń, odzyskanie większej liczby ramek w ustalonym czasie oraz stworzenie nowej metody naukowej. Pierwszy jest problemem implementacyjnym, drugi problemem skuteczności odbiornika, a trzeci wymaga wykazania różnicy względem literatury. Samo użycie GPU, Viterbiego, BCJR, estymatora bayesowskiego lub sieci neuronowej nie stanowi nowości.

Rekomendowany porządek prac to: **pakietowy backend CUDA zachowujący aktualne wyniki → dodatkowa gałąź uwzględniająca niepewność kanału dla audio → koherentny odbiornik GMSK na IQ → ewentualnie uczona metryka detektora.** Wielostacyjna koherentna kombinacja sygnałów pozostaje poza tym zakresem.

Obecna maszyna badawcza nie udostępnia `nvidia-smi`, `nvcc` ani urządzeń `/dev/nvidia*`. Nie oznacza to braku A40 w innym serwerze, lecz brak podstaw do pomiaru jej wydajności w tym środowisku. **Nie wykonano benchmarku CUDA i nie ma zmierzonego przyspieszenia ani dodatkowego zysku telemetrycznego z GPU.**¹

2. Punkt wyjścia: co już działa

Główny wariant progresywny analizuje mono audio po demodulacji FM, w oknach 6 s z krokiem 3 s. Łączy dwa tory filtracji, 160 ustalonych hipotez zegara na tor, dodatkowe konfiguracje Gardnera i sekwencyjną estymację symboli. Modele kanału pochodzą z osobno odebranych, sprawdzonych ramek; dodatkowe gałęzie obejmują estymację bez ramki treningowej i łączenie zgodnych modeli. Wariant główny nie jest tym samym co eksperymentalny odbiornik z wagami pochodzącymi z kodeka.²

W utrwalonym stanie danych z 12 września 2026, 22:52:19 UTC, ukończono pięć torów dla 189 z 266 obserwacji CANVAS. Wariant progresywny uzyskał 4541 unikalnych w obrębie obserwacji ramek UI, wobec 2998 w sumie zbiorów Dire Wolfa i gr-satellites: 1547 dodatków, cztery pominięcia, 1543 netto, czyli 51,47%. Są to powtórne odtworzenia historycznych nagrań, nie świeży niewidziany holdout ani pełny test wszystkich dekoderek SatNOGS. Eksperymentalne warianty z kodekiem i bez kodeka miały identyczne zbiory w każdej ze 189 obserwacji — po 4703 ramki łącznie.³

Średni czas wariantu progresywnego wynosił w tym stanie około 179,3 s na nagranie, a porównywanych programów odpowiednio 3,5 i 3,8 s. Są to czasy na współdzielonym hoście, bez izolacji zasobów i bez wspólnej konwersji audio w czasie pojedynczego toru. Istnieje więc realny problem kosztu obliczeń, ale te liczby nie mówią jeszcze, jaka część kosztu nadaje się do przyspieszenia na GPU.⁴

Osobne wcześniejsze porównanie optymalizacji CPU dało 1,853× przyspieszenia na czterech parach uruchomień dwóch nagrań, przy zgodności sprawdzanych wyników. To pomiar CPU,

¹Telemetry Yield, lokalny audyt kodu i pomiarów, 12 września 2026. Pliki i linie implementacji oraz zakres kontroli sprzętu są wskazane w audycie.

²Telemetry Yield, lokalny audyt kodu i pomiarów, 12 września 2026. Pliki i linie implementacji oraz zakres kontroli sprzętu są wskazane w audycie.

³Telemetry Yield, `summary.json` oraz `observations.json`, utrwalony stan z 12 września 2026, 22:52:19 UTC; materiał lokalny.

⁴Telemetry Yield, `summary.json` oraz `observations.json`, utrwalony stan z 12 września 2026, 22:52:19 UTC; materiał lokalny.

nie prognoza CUDA. Brakuje aktualnego profilu funkcji określającego udział interpolacji, trellis, sortowania parametrów zegara i parsowania ramek w zoptymalizowanym programie.⁵

3. A40: możliwości i ograniczenia

Dokumentacja NVIDIA podaje dla A40 48 GB GDDR6 ECC, przepustowość pamięci 696 GB/s i szczytowe 37,4 TFLOPS FP32. Karta należy do compute capability 8.6; docelową architekturą kompilacji jest sm_86. Są to parametry sprzętu, a nie osiągi dekodera. Wartości Tensor Core dotyczą innych operacji i precyzji niż zwykły kod f64.⁶⁷

Właściwość	Znaczenie dla odbiornika
48 GB pamięci urządzenia	Można przechowywać wiele okien, modeli i historii decyzji; nie trzeba jednak materializować całego archiwum
Wysoka wydajność FP32	Dobra dla filtrów, FFT i modeli uczonych po odrębnej kwalifikacji numerycznej
Niska względna wydajność FP64	Bezpośredni port obecnego f64 nie wykorzysta reklamowanej mocy FP32
Równoległe wątki i warpy	Korzystne dla tysięcy niezależnych prób, słabsze dla jednej krótkiej pętli zależnej od poprzedniego symbolu
PCIe i uruchamianie kerneli	Narzut trzeba amortyzować dużymi partiami; małe zadania często lepiej pozostawić na CPU
Brak MIG	Nie należy projektować izolacji współdzielonych zadań z założeniem tej funkcji

Tabela instrukcji CUDA podaje dla CC 8.6 wydajność 128 wyników FP32 oraz dwóch FP64 na cykl multiprocessora dla odpowiedniej rodziny operacji. Iloraz wynosi 64:1. Przy takim uproszczeniu 37,4 TFLOPS FP32 odpowiada około **0,58 TFLOPS FP64**, nie 37,4; jest to wyprowadzenie z parametrów, nie pomiar. Porównywanie A40 z A100 tylko po nazwie „Ampere” prowadzi do błędnych wniosków, szczególnie dla FP64 i Tensor Cores.⁸

Wniosek inżynierski: A40 jest atrakcyjna do obsługi archiwum i wielu hipotez jednocześnie. Nie należy jednak obiecywać 10×, 50× ani 100× dla całego programu bez prototypu. Szybki kernel może nie skrócić znacząco czasu, jeśli dominują przygotowanie na CPU, parsowanie wszystkich kandydatów, kopiowanie albo małe partie.

4. Co przenieść na GPU

4.1. Najlepsza oś równoległości

Aktualny detektor MLSE ma cztery stany. Odbiornik innovations przy rzędzie modelu szumu 0, 1 lub 2 ma odpowiednio 4, 8 lub 16 stanów. Aktualizacja kolejnego symbolu zależy od poprzednich metryk, więc sama pojedyncza instancja ma ograniczoną równoległość. Natomiast nagrania, okna, frontendy, fazy zegara, modele i współczynniki wzmocnienia tworzą dużą liczbę niezależnych zadań.⁹

⁵Telemetry Yield, lokalny audyt kodu i pomiarów, 12 września 2026. Pliki i linie implementacji oraz zakres kontroli sprzętu są wskazane w audycie.

⁶NVIDIA, A40 Datasheet, marzec 2022, s. 1.

⁷NVIDIA, CUDA GPU Compute Capability, wpis A40, dostęp 12 września 2026.

⁸NVIDIA, CUDA C++ Programming Guide 12.6, tabela 4. Wartość 0,58 jest rachunkiem z parametrów, nie pomiarem.

⁹Telemetry Yield, lokalny audyt kodu i pomiarów, 12 września 2026. Pliki i linie implementacji oraz zakres kontroli sprzętu są wskazane w audycie.

Pierwszy prototyp powinien pakietować gotowe zadania o podobnej długości i liczbie stanów. Warto porównać dwa układy: jeden wątek przechowujący cały mały stan trellis oraz małe podgrupy wątku warp przypisane do stanów jednej instancji. Nie można rozstrzygnąć z samego kodu, który będzie szybszy; zależy to od rejestrów, dostępu do historii decyzji i liczby równoległych prób.

Komponent	Priorytet	Sposób przeniesienia	Granica poprawności
Interpolacja ustalonego zegara	Pierwszy prototyp	Wiele hipotez i okien; wspólne próbki wejściowe	Te same indeksy, brzegi, fazy i kolejność działań
MLSE i innovations	Pierwszy prototyp	Pakiety trellis 4/8/16 stanów	Pełne odtwarzanie ścieżki, te same remisy i końce
NRZI, G3RUH i wykrywanie flag	Drugi etap	Operacje bitowe, pakowanie wyników	Bez utraty kandydatów przy przepełnieniu bufora
FIR	Po profilu kosztów	Równoległe wyjścia, ustalona kolejność sumowania wewnątrz wyjścia	FFT zamiast FIR nie jest automatycznie bitowo równoważne
Ranking zegara, małe regresje	Początkowo CPU	Amortyzacja i stabilność wyboru modeli	Równoległa redukcja może zmienić kolejność hipotez
Pętla Gardnera/Costas	Wiele instancji naraz	Sekwencyjne sprzężenie w każdej instancji	Nie dzielić dowolnie strumienia bez przeniesienia stanu
FFT i wyszukiwanie częstotliwości IQ	Wysoki priorytet toru IQ	Przetwarzanie wielu transformacji i przesunięć	Osobna kwalifikacja względem aktualnej biblioteki FFT
LDPC	Gdy obsługiwany sygnał naprawdę go używa	Wiele słów i węzłów grafu	Ta sama macierz, harmonogram i reguły zakończenia
OGG, manifest, checkpoint, kontrola FCS	Początkowo CPU	Zachowanie istniejącego toru kontroli	Nie ma korzyści z przenoszenia całego systemu plików na GPU

To mapa proponowanego podziału pracy, a nie istniejący backend. NVIDIA zaleca ograniczenie transferów, ciągły dostęp do pamięci i dostateczne wykorzystanie urządzenia; cuFFT ma interfejsy do wielu transformacji w jednej partii.¹⁰¹¹

4.2. Pamięć i przesyłanie danych

Okno 6 s przy 9600 symbolach/s ma orientacyjnie 57 600 symboli, przed uwzględnieniem brzegów i odrzucanych fragmentów. Historia z jednym bajtem na stan i symbol zajmuje około 230 400 bajtów dla czterech stanów lub 921 600 dla szesnastu. Tysiąc prób 16-stanowych to około 0,92 GB samej historii — wykonalne na A40, lecz nie zerowy koszt. To rachunek rozmiaru, nie zmierzona alokacja.¹²

¹⁰NVIDIA, Ampere Tuning Guide, rozdz. 1.2–1.4.

¹¹NVIDIA, cuFFT, PlanMany, przetwarzanie wielu transformacji.

¹²Telemetry Yield, lokalny audyt kodu i pomiarów, 12 września 2026. Pliki i linie implementacji oraz zakres kontroli sprzętu są wskazane w audycie.

Każdy stan ma dwa dopuszczalne poprzedniki, więc decyzję można przechowywać jako jeden bit zamiast bajtu. Daje to ośmiokrotną redukcję samej tablicy wyborów bez skracania historii, pod warunkiem poprawnej rekonstrukcji poprzednika. Podobnie 57 600 decyzji zapisanych jako f64 zajmuje około 460 800 bajtów, a jako bity około 7200 bajtów. Układ buforów i pakowanie mogą więc być równie ważne jak liczba operacji zmiennoprzecinkowych.

Należy wgrywać przygotowane okno raz i używać go w wielu hipotezach, zamiast kopiować osobny sygnał dla każdej fazy. Rozmiar partii powinien zależeć od wolnej pamięci, liczby stanów, długości oraz rezerwy dla pozostałych zadań. Brak miejsca oznacza mniejszą partię lub jawny błąd, a nie odrzucenie części hipotez.

5. Co znaczy „bez utraty dokładności”

5.1. Trzy różne wymagania

Równoważność matematyczna, zgodność bitowa obliczeń i identyczny zbiór odzyskanych ramek to różne rzeczy. Dwa poprawne obliczenia IEEE 754 mogą różnić się wskutek łączenia mnożenia i dodawania, kolejności sumowania czy implementacji funkcji matematycznych. W pobliżu remisu mała różnica zmienia zwycięską ścieżkę.¹³

Obecny kod celowo używa f64 bez fast-math, określonej kolejności rozstrzygania remisów i pełnej historii. Dlatego pierwszy backend powinien pozostawić przygotowanie modeli i ranking na CPU, przenieść ograniczony zestaw działań i sprawdzać zgodność na poziomie decyzji, modeli, identyfikatorów zadań oraz ramek. Dla CUDA C/C++ kontroluje się m.in. kontrakcję FMA przez `--fmad=false`; nie jest to samo w sobie dowodem pełnej zgodności. W kompilatorze Rust→PTX trzeba zweryfikować odpowiedni mechanizm i wygenerowany kod, a nie zakładać, że nazwa typu wystarcza.¹⁴¹⁵

Sprawdzenie CRC na CPU tylko dla kandydatów zwróconych przez przybliżony GPU **nie dowodzi bezstratności**. Jeśli GPU zgubił właściwą ścieżkę, nie zwróci ramki do sprawdzenia. Pewny fallback wymaga odtworzenia pełnej odpowiedniej próby albo udowodnionego certyfikatu numerycznego, nie jedynie prognozy „małej pewności”.

W praktyce należy utrzymywać dwa jawne warianty: backend zgodnościowy z niezmienną polityką oraz oddzielny eksperymentalny backend FP32 lub z inną metryką. Ten drugi może okazać się równie dobry lub lepszy empirycznie, ale jego akceptacja wymaga porównania dodatków i pominięć, nie tylko sumy ramek. Zachowanie kompletnej gałęzi CPU i dodanie GPU gwarantuje zachowanie jej zbioru na poziomie sumy poprawnie ukończonych wyników, lecz nie usuwa kosztu tej gałęzi CPU.

5.2. Istniejące równoległe algorytmy Viterbiego

Publikacje opisują pakietowe dekodery Viterbiego na GPU, optymalizację historii i wersje wykorzystujące Tensor Cores. Prace Mohammadidoosta i Hashemiego dotyczą dekodowania kodów splotowych, nie bezpośrednio obecnego czterostanowego modelu kanału Telemetry Yield. Wersja z Tensor Cores bada precyzję i pokazuje pogorszenie przy zbyt niskiej precyzji akumulacji; nie dostarcza gwarancji zachowania wyników naszego f64.¹⁶¹⁷

Istnieje również temporalna równoległość przez składanie operatorów i parallel scan w modelach Markowa. Hassan, Särkkä i García-Fernández opisują taką konstrukcję dla wnioskowania i ścieżki

¹³NVIDIA, Floating-Point Computation, rozdz. 5.5.

¹⁴Telemetry Yield, lokalny audyt kodu i pomiarów, 12 września 2026. Pliki i linie implementacji oraz zakres kontroli sprzętu są wskazane w audycie.

¹⁵NVIDIA, NVCC 12.6, `--fmad`.

¹⁶Mohammadidoost i Hashemi, GPU Viterbi dla kodów splotowych, 2020.

¹⁷Mohammadidoost i Hashemi, Viterbi na Tensor Cores, 2020, pełny tekst s. 11, sekcja o precyzji.

MAP, a praca EUSIPCO 2023 rozwija równoległość Viterbiego w czasie. To ważny kontrargument wobec twierdzenia, że Viterbi „nie może działać równolegle”. Wydajność zależy jednak także od liczby stanów i kosztu składania operatorów.¹⁸¹⁹

Dla obecnego bardzo małego trellis i dużej liczby niezależnych hipotez zwykle pakietowanie jest prostszym punktem startowym. Parallel scan zmienia grupowanie działań zmiennie-przecinkowych, a więc wymaga osobnej kwalifikacji. Skrócone odtwarzanie ścieżki i nakładające się kafle nie powinny być wprowadzane jako „dokładnie to samo” bez dowodu poprawności brzegów dla danego wariantu.

6. Kandydat dla OGG/WAV: niepewność modelu kanału

6.1. Uzasadnienie

Obecny tor przenosi oszacowany lokalny model z poprawnie odebranej ramki na inne okno. Model może być niedokładny, przestać pasować z czasem albo powstać z małej liczby użytecznych symboli. Obecna bliskość czasowa i ocena jakości nie są tym samym co jawny rozkład niepewności parametrów.²⁰

Hipoteza badawcza: dodatkowa metryka uwzględniająca zarówno zakłócenia, jak i niepewność przeniesionego modelu poprawi odzysk na trudnych oknach, nie wymagając błędnie idealnej znajomości kanału. Estymacja bayesowska i per-survivor processing są wcześniejszymi rozwiązaniami tej ogólnej klasy problemów. Nie można ogłosić nowości samej „niepewności kanału”.²¹²²

6.2. Proponowana, jeszcze niezaimplementowana konstrukcja

Pierwsza wersja powinna ograniczyć się do modelu bez autoregresyjnego szumu, aby nie mieszać kilku zmian jednocześnie. Niech θ obejmuje bias i trzy współczynniki kanału, a ϕ_n odpowiednie symbole i stałą. Na podstawie oddzielonych ramek kotwiczących estymuje się średnią m_n i skalibrowaną kowariancję P_n parametrów; dodatkowo $R_n > 0$ opisuje wariancję reszty. Kandydat metryki ma postać

$$\mu_n = \phi_n^T m_n, \quad S_n = R_n + \phi_n^T P_n \phi_n,$$

$$\ell_n = \frac{(y_n - \mu_n)^2}{S_n} + \log S_n.$$

Jest to standardowo motywowana gaussowska lokalna metryka predykcyjna, proponowana tutaj jako dodatkowa gałąź. Człon $\log S_n$ jest konieczny: bez niego zbyt duża deklarowana niepewność sztucznie zmniejsza karę za błąd. Kowariancja nie powinna być bezrefleksyjnie utożsamiana z macierzą odwrotną z regresji ridge; jej skala i pokrycie wymagają kalibracji na oddzielonych danych.

Implementacja musi wymuszać symetrię i dodatnią półokreśloność P_n , dodatnią dolną granicę wariancji R_n oraz skończoną, dodatnią wartość S_n . Średnia m_n i kowariancja P_n mają być wyznaczone z danych kotwiczących i ustalone dla danej próby, niezależnie od rozważanej ścieżki symboli w oknie docelowym. Dopiero przy tym warunku oraz lokalnej zależności od trzech

¹⁸Hassan, Särkkä i García-Fernández, Temporal Parallelization of Inference in Hidden Markov Models, 2021.

¹⁹On the Temporal Parallelisation of the Viterbi Algorithm, EUSIPCO 2023.

²⁰Telemetry Yield, lokalny audyt kodu i pomiarów, 12 września 2026. Pliki i linie implementacji oraz zakres kontroli sprzętu są wskazane w audycie.

²¹Channel Decoding with a Bayesian Equalizer, 2010, pełny tekst s. 1–3.

²²R. Raheli, Introduction to Per-Survivor Processing, 2004, materiały autora; artykuł odniesienia z 1995: DOI.

symboli można zachować czterostanowy trellis; adaptacja parametrów osobno wzdłuż każdej ścieżki byłaby innym, bardziej złożonym odbiornikiem.

Ważne ograniczenie matematyczne: wspólny nieznany kanał wywołuje zależności między resztami różnych symboli. Suma lokalnych metryk po marginalizacji pojedynczych próbek nie jest automatycznie dokładną marginalizacją całej sekwencji. Powyższa wersja jest przybliżeniem i powinna być tak nazwana. Bardziej złożone wnioskowanie wspólne, model stanu kanału albo per-survivor processing to kolejne porównania, nie ukryte założenie dokładności.

Ruch modelu w czasie można badać przez ograniczony model stanu, np. $\theta_{n+1} = \theta_n + w_n$, z parametrami wyznaczanymi poza oknem docelowym. Początkowo należy rozdzielić wersję stałą z niepewnością od wersji czasowo zmiennej. Odrzucona kalibracja wyłącza wyłącznie nową gałąź, a wyniki dotychczasowego odbiornika pozostają zachowane. Ramki odzyskane nową metodą nie powinny automatycznie stawać się kolejnymi kotwicami treningowymi.

6.3. Rola A40 i potencjalny wkład

GPU pozwala równocześnie testować więcej zgodnych modeli i realizacji trellis. Na początek nie są potrzebne Tensor Cores: model ma mało parametrów, a główna równoległość wynika z liczby prób. Jeżeli lokalna metryka zależy tylko od obecnego i dwóch sąsiednich symboli, nie trzeba zwiększać liczby stanów wyłącznie z powodu samego członu niepewności. Złożony model wspólny lub autoregresyjny może już wymagać dodatkowej pamięci stanu.

Potencjalny wkład publikacyjny to **ściśle określony, odporny na przeciek transfer niepewności kanału między ramkami historycznego audio, z addytywną gałęzią odbioru i pomiarem koszt–zysk**. To hipoteza o wkładzie, nie potwierdzone pierwszeństwo. Należy wykazać różnicę wobec bayesowskich equalizerów, per-survivor processing i już wdrożonego ważenia modeli oraz porównać je w identycznych warunkach.

Najważniejszy test obejmuje osobno: stały model punktowy, stały model z niepewnością, czasowo zmienny model punktowy i czasowo zmienny model z niepewnością. Identyczne banki zegara i reguły przyjmowania ramek pozwolą ustalić, czy działa nowa metryka, czy tylko zwiększenie liczby prób. Wynik zerowy również jest rozstrzygnięciem.

7. Kierunki dla IQ i miękkiej detekcji

7.1. Koherentny GSMK z modelem CPM

Surowe IQ zachowuje informację o fazie, którą tor audio po FM utracił. Dla IQ warto rozważyć dopasowanie impulsów oparte na dekompozycji Laurenta, estymację częstotliwości i jej dryfu, synchronizację oraz sekwencyjny odbiornik uwzględniający pamięć modulacji. Autor gr-satellites, Daniel Estévez, opisał w 2025 r. działający odbiornik koherentny GSMK oparty na tej dekompozycji na rzeczywistych danych BGM-1. To bezpośrednio istotny punkt odniesienia, nie nowość do przypisania Telemetry Yield.²³

Własny eksperyment może porównać dominujący impuls Laurenta z dodatkowymi składowymi, kilka poprawnych hipotez szerokości impulsu oraz kontrolowany bank częstotliwości/dryfu. A40 może równolegle obliczać filtry dopasowane i metryki tych hipotez. Parametry kodowania, precodera i synchronizacji muszą odpowiadać nadajnikowi; sama etykieta GSMK nie wystarcza do ustalenia całego protokołu.

To dobry kandydat na wzrost skuteczności IQ, ale nie na rzetelne hasło „odzyskujemy fazę z OGG”. Rzetelny test musi porównać tory na tej samej próbce IQ i z tym samym wycinkiem czasu. Wynik koherentny z IQ nie powinien być przedstawiany jako porównanie wyłącznie algorytmów, gdy baseline otrzymał tylko stratne audio.

²³D. Estévez, Decoding BGM-1 GSMK telemetry, marzec 2025.

7.2. BCJR i rzeczywista korekcja błędów

Aktualny MLSE zwraca twarde decyzje symbolowe. BCJR lub jego odpowiednio opisane przybliżenie może dostarczyć prawdopodobieństwa bitów, użyteczne dla istniejącego w nadajniku kodowania. Literatura bayesowskiego equalizera pokazuje, że lepsza kalibracja prawdopodobieństw może mieć istotniejsze znaczenie po LDPC niż po samym twardym decyzyjowaniu.²⁴

Nie wynika z tego, że BCJR zawsze odzyska więcej niekodowanych ramek AX.25 niż MLSE. Optymalizacja decyzji bitowej i całej sekwencji to odmienne kryteria, a CRC nie tworzy brakującego kodu LDPC. Należy zachować dotychczasową gałąź i oceniać dodatkowy zysk, kalibrację miękkich informacji oraz pominięcia. Dla CCSDS potrzebne są konkretne schematy kodowania, randomizacji i ramek użyte przez misję.

GPU ma dobre zastosowanie do wielu słów LDPC, lecz gotowy moduł 5G nie jest ogólnym dekoderym CCSDS. Dokumentacja NVIDIA Aerial wyraźnie dotyczy łańcucha LDPC 5G. Istniejący kod Telemetry Yield ma wybrane presety, nie uniwersalne AR4JA, DVB-S2 czy kodowanie splotowe z dowolnym puncturingiem. Uzupełnienie zgodności z realnym nadajnikiem może dać większy praktyczny efekt niż przyspieszanie niewłaściwego profilu.²⁵²⁶

8. Czy użyć sieci neuronowej

Najbardziej uzasadniona ścieżka to niewielki model poprawiający lokalną metrykę, nie generator całych ramek. ViterbiNet już łączył uczoną część detektora z klasyczną strukturą Viterbiego, a Meta-ViterbiNet badał adaptację do zmiennego kanału. Te prace ograniczają dopuszczalne deklaracje nowości.²⁷²⁸

Potencjalny eksperyment Telemetry Yield może uczyć resztową korektę analitycznej metryki z ograniczoną amplitudą i powrotem do klasycznego toru poza rozkładem treningowym. Trening powinien używać wiarygodnych etykiet symbolowych, kontrolowanych symulacji oraz całkowicie oddzielonych nagrań do oceny. Próbkę z tej samej transmisji, stacji albo powtarzającą się zawartość nie mogą swobodnie trafiać po obu stronach podziału.

Nie należy nazywać etykietą prawdy każdej ramki zaakceptowanej przez dowolną eksperymentalną gałąź. Samonapędzające się uczenie na błędnych decyzjach może utrzymywać błąd. Dotychczasowy brak dodatkowego zysku wariantu codec-enabled jest argumentem za kontrolowanymi, małymi eksperymentami, a nie za założeniem, że każdy dodatkowy model będzie skuteczniejszy.

DeepRx opisuje pełniejszy odbiornik uczony dla innego łańcucha, a „Machine LLRning” uczenie miękkiej demodulacji. Nie są gotowymi zamiennikami odbiornika post-FM GMSK/AX.25 z archiwum. Ich przydatność trzeba ograniczyć do odpowiednich elementów modelu i warunków transmisji.²⁹³⁰

Osobny model oceniający waterfall może sterować kolejką pracy, ale odrzucanie na tej podstawie nagrań jest potencjalnym źródłem strat. Nawet bardzo czuły selektor wymaga oceny fałszywych odrzuceń i prób kontrolnych z nisko ocenionych okien. Taki selektor nie jest bezstratnym przyspieszeniem samego dekodera.

²⁴Channel Decoding with a Bayesian Equalizer, 2010, pełny tekst s. 1–3.

²⁵NVIDIA, Aerial LDPC 5G.

²⁶Telemetry Yield, lokalny audyt kodu i pomiarów, 12 września 2026. Pliki i linie implementacji oraz zakres kontroli sprzętu są wskazane w audycie.

²⁷Shlezinger i in., ViterbiNet, IEEE TWC, 2020.

²⁸Raviv i in., Meta-ViterbiNet, 2021.

²⁹DeepRx, 2020.

³⁰„Machine LLRning”, 2019.

9. CUDA przy zachowaniu Rust

Nie ma konieczności przepisywania sterowania, profili, walidacji i checkpointów poza Rust. Część urządzeniowa wymaga jednak narzędzi przeznaczonych do GPU; istniejącego programu z Rayon, alokacjami i systemem plików nie uruchamia się automatycznie na karcie.

Ścieżka	Co zapewnia	Ocena dla projektu
Rust host + cudarc + PTX/CUDA kernels	Obsługa sterownika, pamięci i uruchamiania kerneli; opcjonalne NVRTC	Wygodna warstwa sterująca; kernel w CUDA C++ nadal oznacza drugi język
Rust-CUDA	Kompilacja i narzędzia dla kodu urządzenia w Rust	Kandydat dla małego, wydzielonego backendu; przypięty toolchain i kwalifikacja
NVlabs cuda-oxide	Własny backend Rust→PTX, kod SIMT w Rust	Aktualny kandydat zgodny z celem jednego języka; sprawdzić A40 i semantykę FP64 na prototypie

cudarc jest wrapperem toolkit/driver, a jego przykład NVRTC kompiluje tekst CUDA C++; nie jest to automatycznie kompilator kerneli Rust. Rust-CUDA używa przypiętego narzędziowego zestawu. Aktualny README cuda-oxide opisuje kompilację Rust→PTX, zależności od przypiętego nightly oraz CUDA 13 i sterownika R580+. ³¹³²³³

Autorzy oznaczają cuda-oxide jako projekt **we wczesnej fazie alpha**, z możliwymi błędami, niepełnym wsparciem funkcji i zmianami API. Jest to kandydat do izolowanego prototypu, nie uzasadnienie deklaracji gotowości produkcyjnej ani automatyczny wybór dla całego odbiornika. ³⁴

Dokumentacja cuda-oxide zawiera ścieżki dla sm_86, ale różne wersje materiałów instalacyjnych wskazują różne minima sterownika. Dlatego trzeba przypiąć jeden commit i odpowiadające mu wymagania, zamiast mieszać instrukcje ze starszej strony i bieżącej gałęzi. Nie ma podstaw do instalowania nowego sterownika na działającym hoście badawczym tylko w celu przygotowania raportu. ³⁵

Rekomendowany kontrakt to oddzielny moduł kerneli z prostymi buforami i opisami zadań. Backend CPU pozostaje dostępny bez CUDA. Wybór urządzenia, precyzji, pliku PTX/cubin, sterownika i polityki obliczeń trafia do manifestu. Zmiana backendu nie powinna pozwalać na niejawne kontynuowanie starego checkpointu tak, jakby był to ten sam eksperyment.

Na początek warto preferować mały kernel w Rust generujący standardowy kod SIMT dla sm_86, bez funkcji specyficznych dla Hopper/Blackwell. Jeżeli wybrany kompilator nie odtworzy wymaganego zachowania numerycznego, należy to uznać za wynik kwalifikacji, nie ukrywać przejściem na FP32. Alternatywą pozostaje jawnie zaakceptowany mały komponent CUDA C++ albo osobny eksperymentalny profil numeryczny.

³¹Projekt cudarc, README, dostęp 12 września 2026.

³²Rust-GPU, Rust-CUDA getting started, dostęp 12 września 2026.

³³NVlabs, cuda-oxide, bieżący README, dostęp 12 września 2026.

³⁴NVlabs, cuda-oxide, bieżący README, dostęp 12 września 2026.

³⁵NVlabs, cuda-oxide supported features i launching kernels, dostęp 12 września 2026. Minima zależności należy sprawdzić dla jednego przypiętego commitu.

10. Plan eksperymentu i kryteria decyzji

Etap A: profil i zgodność

Należy zamrozić CPU oracle oraz nagrania rozwojowe, zebrać czasy poszczególnych etapów, a po udostępnieniu A40 zmierzyć transfer, uruchamianie i kernel oddzielnie. Pierwszy backend obejmuje interpolację oraz 4/8/16-stanowe trellis; ranking i modele pozostają na CPU. CUDA profiler pomaga odróżnić pracę urządzenia od kolejek i kopiowania, ale sam czas kernela nie zastępuje czasu od pliku do zweryfikowanych ramek.³⁶

Warunek przejścia: kompletna zgodność uzgodnionych wyników, brak przepełnień i pomijania zadań, prawidłowe wznowienie oraz faktycznie mniejszy czas end-to-end w istotnym trybie. Należy porównać partie 1, 8, 64 i większe, zimny i ciepły start, krótkie oraz długie nagrania, pełny bank i budżet 3/60 s. Większa przepustowość archiwum nie dowodzi lepszej latencji pojedynczego nagrania.

Etap B: dodatkowe możliwości

Przy niezmiennych danych i regułach walidacji porównać stały budżet czasu dla CPU i GPU, rejestrując rzeczywiście ukończone hipotezy. Dopiero osobna zamrożona polityka może dodawać dokładniejszy model kanału, większą pamięć lub BCJR. Zwiększenie banku i zmiana metryki muszą być dwiema niezależnymi osiami eksperymentu.

W każdym porównaniu trzeba raportować dodatkowe i utracone zbiory ramek, bajty użytecznej zawartości według jawnej definicji, odsetek obserwacji ze wzrostem, grupę z etykietą obecności sygnału, nieukończone zadania, p50/p95 latencji, całkowitą przepustowość, zużycie CPU oraz szczyt RAM/VRAM. Jeśli oceniana jest energia, trzeba zmierzyć energię całego porównywanego procesu, a nie porównywać samego limitu mocy GPU z czasem CPU.

Etap C: test naukowy

Obecny historyczny zbiór po ujawnieniu wyników nie jest niewidzianym holdoutem dla kolejnego algorytmu. Nową metodę trzeba zamrozić przed oceną na oddzielonych przelotach, danych, a dla szerszych wniosków także misjach i torach odbiorczych. Wnioskowanie statystyczne powinno uwzględnić zależności między stacjami obserwującymi ten sam przelot oraz powtarzające się ramki.

Kontrolne ujemne obejmują szum, brak transmisji, błędny profil, zakłócenia i kontrolowane uszkodzenia sygnału. CRC/FCS należy sprawdzać niezależnie, ale nie traktować go jako kryptograficznego potwierdzenia źródła. Większy bank hipotez zwiększa także liczbę okazji do przypadkowego zaakceptowania kandydata.

Granice oczekiwanego przyspieszenia

Dla udziału p czasu nadającego się do akceleracji i przyspieszenia tej części s , uproszczone prawo Amdahla daje $S = 1/((1 - p) + p/s)$, jeszcze bez dodatkowych transferów. Jeśli przykładowo $p = 0,8$ i $s = 20$, całość przyspiesza najwyżej około $4,17\times$ w tym modelu. Jest to **ilustracja arytmetyczna, nie prognoza dla Telemetry Yield**. Dopóki p nie zostanie zmierzone, nie należy podawać obiecanego czasu A40.

11. Rekomendacja końcowa

Warto przygotować A40 jako opcjonalny akcelerator do przetwarzania partii, zachowując Rust jako warstwę sterującą i docelowo język kerneli. Najbezpieczniejszy pierwszy rezultat to szybsze

³⁶NVIDIA, Nsight Systems User Guide.

wykonanie identycznej, ograniczonej części istniejącego odbiornika z mierzalną zgodnością. Jest to samodzielnie użyteczna optymalizacja stacji, nawet jeśli nie będzie nowym algorytmem.

Najbardziej uzasadnioną nową gałęzią audio jest następnie jawne uwzględnienie niepewności przeniesionego modelu i kontrolowanej zmienności kanału. Dla IQ większy potencjał może mieć zgodny z nadajnikiem odbiornik koherentny GMSK z dokładniejszym modelem CPM i miękkimi informacjami dla rzeczywistej korekcji błędów. Ocena „większy potencjał” jest rekomendacją inżynierską, a nie zmierzonym wynikiem.

Nie ma obecnie podstaw do deklaracji, że ta metoda jest pierwsza na świecie, że A40 da określony mnożnik przyspieszenia albo że GPU samo zwiększy liczbę ramek. Jest natomiast konkretny, wykonalny plan sprawdzenia tych możliwości bez zanieczyszczania trwającego porównania publikacyjnego.

Źródła

Materiały internetowe sprawdzono 12 września 2026. Publikacje dotyczą różnych kodów, moduacji i sprzętu; ich liczby wydajności nie zostały przeniesione na Telemetry Yield. Poniższy wykaz obejmuje źródła pierwotne oraz jawnie wskazane lokalne dane implementacyjne.

1. NVIDIA. NVIDIA A40 Datasheet, marzec 2022, s. 1. Parametry A40, precyzje i brak MIG.
2. NVIDIA. CUDA GPU Compute Capability. A40 jako CC 8.6.
3. NVIDIA. CUDA C++ Programming Guide, 12.6, Arithmetic Instructions, tabela 4. Wydajność operacji FP32/FP64.
4. NVIDIA. Ampere Tuning Guide, rozdz. 1.2–1.4. Równoległość i dostęp do pamięci; rozróżnienia architektur.
5. NVIDIA. Floating-Point Computation, rozdz. 5.5. Arytmetyka, FMA i deterministyczność.
6. NVIDIA. CUDA Compiler Driver NVCC, 12.6. Kontrakcja FMA.
7. NVIDIA. cuFFT documentation, interfejsy PlanMany. Przetwarzanie partii FFT.
8. A. Mohammadidoost, M. Hashemi. High-Throughput and Memory-Efficient Parallel Viterbi Decoder for Convolutional Codes on GPU, 2020. Pełny tekst, struktura i historia dekodowania.
9. A. Mohammadidoost, M. Hashemi. High-Throughput Parallel Viterbi Decoder on GPU Tensor Cores, 2020, szczególnie sekcja o precyzji i rys. 13. Wyniki zależne od precyzji.
10. S. Hassan, S. Särkkä, Á. F. García-Fernández. Temporal Parallelization of Inference in Hidden Markov Models, 2021. Równoległe sum-product i Viterbi/MAP.
11. On the Temporal Parallelisation of the Viterbi Algorithm, EUSIPCO 2023. Alternatywna temporalna równoległość.
12. Channel Decoding with a Bayesian Equalizer, 2010. Niepewność CSI, BCJR i efekt po LDPC.
13. R. Raheli. Introduction to Per-Survivor Processing, materiały autorskie CNIT, 2004; bibliografia zawiera R. Raheli, A. Polydoros, C.-K. Tzou, „Per-survivor processing: a general approach to MLSE in uncertain environments”, 1995, DOI 10.1109/26.380054. Zakres koncepcji; pełny artykuł czasopisma nie został wykorzystany jako źródło niezweryfikowanych szczegółów.
14. D. Estévez. Decoding BGM-1 GMSK telemetry, marzec 2025. Autorski odbiornik koherentny, Laurent, rzeczywiste IQ.
15. N. Shlezinger, N. Farsad, Y. C. Eldar, A. J. Goldsmith. ViterbiNet: A Deep Learning Based Viterbi Algorithm for Symbol Detection, preprint 2019; autorski tekst publikacji IEEE TWC, 2020.
16. T. Raviv i in. Meta-ViterbiNet: Online Meta-Learned Viterbi Equalization for Non-Stationary Channels, 2021.
17. DeepRx: Fully Convolutional Deep Learning Receiver, 2020. Kontekst odmiennych odbiorników uczonych.
18. „Machine LLRning”: Learning to Softly Demodulate, 2019. Uczone miękkie decyzje.

19. NVIDIA. Aerial, LDPC 5G API. Zakres cuPHY nie jest ogólnym CCSDS.
20. Projekt cudarc. Repozytorium i README. Sterownik/NVRTC, a nie automatyczna kompilacja Rust na GPU.
21. Rust-GPU. Rust-CUDA — getting started. Przypięty toolchain i kod urządzeniowy Rust.
22. NVlabs. cuda-oxide README, obsługiwane funkcje, uruchamianie kerneli. Rust→PTX, wymagania i ścieżka sm_86.
23. NVIDIA. Nsight Systems User Guide. Rozdzielenie aktywności CPU, CUDA i transferów.
24. Telemetry Yield. Lokalny audyt kodu CUDA, 12 września 2026; Rust `sequence.rs`, `innovation.rs`, `adaptive.rs`, `progressive.rs`, `dsp.rs`, `protocol.rs`, `psk.rs`, `fec_ldpc.rs`, miejsca podane w audycie.
25. Telemetry Yield. Utrwalone metryki publikacyjne i zbiory per obserwacja, cutoff 2026-09-12 22:52:19 UTC. Lokalny dostęp; nie jest to publiczny DOI zbioru.